

税务发票聚合查询服务（swfp）· API 接口文档与使用手册

面向接入方（客户）技术与运维人员的对外接口说明。
版本：swfp（税务发票聚合） | 通信：HTTP + JSON | 编码：UTF-8

说明：本服务采用统一的请求信封与 MD5 加签方式，响应分为 `head`（网关头部）与 `body`（业务结果）两部分。其余内部处理细节调用方无需关心。
关键特性：本服务为企业维度查询——凭企业统一社会信用代码（`creditCode`）一次查询，聚合返回该企业的**发票数据（两部分）与税务数据（两部分）共四个数据段**；四段结果合并为一个 JSON 字符串置于 `body.result.range`，由调用方自行解析（见「3.1.5 result.range 结构说明」）。若部分数据段获取失败，返回业务码 `002`（部分数据源成功，**不计费**），已成功的数据段照常返回。

一、接入必读

1.1 适用范围

本文档适用于接入本平台「税务发票聚合查询服务（swfp）」的第三方产品技术人员与日常运维人员。适用业务：企业信贷风控、供应链金融、企业经营状况评估等需要企业发票与税务数据的场景。

1.2 接入须知

- 1. 正式访问域名在接入时由我方商务提供。
- 2. 接入前需先申请开通账户，由我方分配 `appKey` 与 `appSecret`（加签密钥）。
- 3. 本服务依据**企业授权**开展查询：调用方应确保已依法取得被查询企业的有效授权。

1.3 接口说明

项目	说明
请求方式	POST（查得数查询为 GET）
通信协议	HTTP
数据格式	请求体与响应体均为 JSON
字符编码	UTF-8
超时时间	4 秒（单次查询内部 并发聚合四个数据段 ，建议客户端读超时 ≥ 6 秒）
HTTP 状态码	恒为 200 ；业务结果与错误均通过响应体的 <code>head.errorCode</code> / <code>body.code</code> 表达
签名	调用时需对 <code>body</code> 中的业务参数 + 我方分配的 <code>appSecret</code> 进行 MD5 加签，详见「二、加签」

1.4 环境说明

- **正式环境**：使用正式账户，调用已开通接口，**按查得成功条数计费**（见「五、计费说明」）。
- **测试环境**：使用测试账户，返回挡板/联调数据。
- 正式账户仅适用于正式环境，测试账户仅适用于测试环境。

二、鉴权与加签

所有业务接口共用同一套请求信封与鉴权方式。

2.1 请求信封（顶层参数）

参数	示例	类型	必填	说明
appKey	y890swfp	String	是	我方分配给客户的公开标识
sign	0528999dd55c025b8f36fc72dceb1f63	String	是	对 body 业务参数的 MD5 签名（见 2.3）
encryptionType	1	int	否	参数加密类型，1 = 明文（默认）
body	{...}	Object	是	业务请求体，见各接口定义

注意：appKey / sign / encryptionType 不参与签名计算。

2.2 鉴权校验顺序

网关按以下顺序校验，任一失败立即返回对应 head.errorCode（直接拒绝，不计费）：

1. appKey 是否存在 → 否则 505001
2. appKey 是否匹配到账户 → 否则 505004
3. 账户是否有效（启用且在有效期内） → 否则 505007
4. 签名是否正确 → 否则 505002

2.3 加签方式

1. 取出 body 中**所有非空的业务参数**（不含文件/字节流类型，不含值为空的参数）。
2. 按参数名（key）的 **ASCII 升序**排序；首字符相同则依次比较后续字符。
3. 将排序后的参数按 参数名 参数值 直接拼接，最后追加 appSecret，得到**待签名串**。
4. 对待签名串做 **MD5**，取 **32 位小写十六进制**字符串，赋值给 sign。

示例：body = { "creditCode": "92500233MA60R5KW8M" }，appSecret = "<你的密钥>"。

待签名串为：

```
creditCode92500233MA60R5KW8M<你的密钥>
```

`sign = MD5(待签名串)` 的小写十六进制值。

提示：拼接顺序由 key 的 ASCII 决定，请勿写死字段顺序；新增字段时排序会自动变化。

2.4 加签代码示例

Java

```
public static String sign(Map<String, String> bodyParams, String appSecret) throws Exception {
    StringBuilder sb = new StringBuilder();
    List<String> keys = new ArrayList<>(bodyParams.keySet());
    Collections.sort(keys); // ASCII 升序
    for (String k : keys) {
        String v = bodyParams.get(k);
        if (v == null || v.isEmpty()) continue; // 剔除空值
        sb.append(k).append(v);
    }
    sb.append(appSecret);
    MessageDigest md = MessageDigest.getInstance("MD5");
    byte[] digest = md.digest(sb.toString().getBytes(StandardCharsets.UTF_8));
    StringBuilder hex = new StringBuilder();
    for (byte b : digest) hex.append(String.format("%02x", b));
    return hex.toString();
}
```

Python

```
import hashlib

def sign(body_params: dict, app_secret: str) -> str:
    parts = []
    for k in sorted(body_params.keys()):          # ASCII 升序
        v = body_params[k]
        if v is None or v == "":
            continue                             # 剔除空值
        parts.append(f"{k}{v}")
    raw = "".join(parts) + app_secret
    return hashlib.md5(raw.encode("utf-8")).hexdigest() # 小写 hex
```

Go

```
func sign(body map[string]string, appSecret string) string {
    keys := make([]string, 0, len(body))
    for k, v := range body {
        if v != "" { // 剔除空值
            keys = append(keys, k)
        }
    }
    sort.Strings(keys) // ASCII 升序
    var sb strings.Builder
    for _, k := range keys {
        sb.WriteString(k)
        sb.WriteString(body[k])
    }
    sb.WriteString(appSecret)
    sum := md5.Sum([]byte(sb.String()))
    return hex.EncodeToString(sum[:]) // 小写 hex
}
```

三、接口列表

3.1 税务发票聚合查询 (swfp)

项目	内容
路径	POST /v1/openapi/zlx/querySrmxSWFP
完整地址	http://www.aiszcloud.cn:8080/v1/openapi/zlx/querySrmxSWFP
鉴权	appKey + MD5 签名（见第二章）

凭企业统一社会信用代码一次查询，并发聚合该企业的**发票数据聚合 (part1/part2)** 与**税务数据聚合 (part1/part2)** 四个数据段，合并返回。

3.1.1 请求 body 参数

参数	示例	类型	必填	说明
creditCode	92500233MA60R5KW8M	String	是	企业统一社会信用代码（18 位）

参数格式非法（非 18 位统一社会信用代码格式）将返回 `head.errorCode = 505062`，直接拒绝，不计费。

3.1.2 请求示例

```
{
  "encryptionType": 1,
  "appKey": "y890swfp",
  "sign": "0528999dd55c025b8f36fc72dceb1f63",
  "body": {
    "creditCode": "92500233MA60R5KW8M"
  }
}
```

3.1.3 响应结构

响应分为 head（网关头部）与 body（业务结果）两部分：

head 字段：

参数	示例	类型	说明
errorCode	0	String	网关返回码。0 = 成功（含查得/查无/部分成功）；非 0 = 网关级错误，此时无 body
errorMsg	success	String	返回描述
logId	a1b2c3...	String	全链路追踪 ID，排障/对账时请提供
time	1280	Number	服务处理耗时（毫秒）
timestamp	1718456789012	Number	响应时间戳（毫秒）

body 字段（仅 head.errorCode = 0 时返回）：

参数	示例	类型	说明
code	001	String	业务结果码。001 = 四段全部应答且有查得【计费】；999 = 全部查无【不计费】；002 = 部分数据段成功【不计费】
msg	成功	String	业务描述
reqid	1kf9x2...	String	本次请求流水号
uid	0140142026...	String	交易流水号（对账用）
result	{...}	Object	业务内容，code = 001 或 002 时存在
result.range	"{"invoice1\": {...},...}"	String	四个数据段合并结果的 JSON 字符串，调用方需 JSON.parse 后使用，结构见 3.1.5

3.1.4 响应示例

① 查得数据 (计费)

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d4", "time": 1280, "t": 1280 },
  "body": {
    "code": "001",
    "msg": "成功",
    "reqid": "1kf9x2ab",
    "uid": "01401420260708995730278800",
    "result": {
      "range": "{\\"invoice1\\":{\\"status\\":\\"ok\\",\\"rawStatus\\":\\"4\\",\\"data\\":{\\"nsrfpxx\\"
    }
  }
}
```

② 全部查无 (不计费)

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d5", "time": 960, "timeStamp": 151401420260708995730278900 },
  "body": {
    "code": "999",
    "msg": "查无结果",
    "reqid": "lkf9x2ac",
    "uid": "01401420260708995730278900"
  }
}
```

③ 部分数据段成功（不计费，已成功的段照常返回）

```
{
  "head": { "errorCode": "0", "errorMsg": "success", "logId": "a1b2c3d6", "time": 1420, "t": 1420 },
  "body": {
    "code": "002",
    "msg": "部分数据源成功",
    "reqid": "1kf9x2ad",
    "uid": "01401420260708995730279000",
    "result": {
      "range": "{\\"invoice1\\":{\\"status\\":\\"ok\\",\\"rawStatus\\":\\"4\\",\\"data\\":{...}},\\"invoice2\\":{\\"status\\":\\"ok\\",\\"rawStatus\\":\\"4\\",\\"data\\":{...}}}"
    }
  }
}
```

④ 网关级错误 (无 body)

```
{
  "head": { "errorCode": "505002", "errorMsg": "账号信息异常", "logId": "a1b2c3d7", "time":
}
```

3.1.5 result.range 结构说明

result.range 是四个数据段合并结果的 **JSON 字符串**。调用方应先对该字符串做一次 JSON.parse / 反序列化，得到含四个固定键的对象：

段名	数据内容
invoice1	发票数据聚合（第一部分）：进销项发票汇总、开票明细等
invoice2	发票数据聚合（第二部分）：发票补充维度数据
tax1	税务数据聚合（第一部分）：纳税申报、税款缴纳等
tax2	税务数据聚合（第二部分）：税务补充维度数据

每段的结构：

字段	类型	说明
status	String	该段获取状态：ok = 查得（带 data）；empty = 查无；error = 该段数据源异常（带 error）
rawStatus	String	原始状态码（备查，4 = 有结果，1 = 无结果）
data	Object/Array	该段明细数据（仅 status = ok 时存在），已解码为明细本体：发票段含 nsrfpxx 等节点、税务段含 nsrswxx 等节点
error	String	失败原因摘要（仅 status = error 时存在）

说明：data 的具体字段可能随数据能力升级扩展，本服务保证**明细本体完整透出**（无需再做任何解码处理）。建议调用方按「存在即解析、缺失即忽略」的方式做容错处理，避免因后续新增字段导致解析失败。

解析示例（JavaScript）：

```
``js
const range = JSON.parse(resp.body.result.range);
for (const key of ["invoice1", "invoice2", "tax1", "tax2"]) {
  const sec = range[key];
  if (sec.status === "ok") {
    console.log(key, "查得:", sec.data);
  } else if (sec.status === "empty") {
    console.log(key, "查无");
  } else {
```

```
console.log(key, "获取失败:", sec.error);
}
}
...
```

3.2 成功查得数查询（扩展接口）

查询本账户累计成功查得数据的次数，用于客户侧自助监控。不返回额度上限/剩余量（本服务无额度限制）。

项目	内容
路径	GET /v1/openapi/zlx/quotaSWFP
鉴权	与主接口一致（请求体中携带 appKey + sign 信封；body 可为 {}，此时 sign = MD5(appSecret) ）。

响应示例

```
{
  "errorCode": "0",
  "errorMsg": "success",
  "status": "ACTIVE",
  "serviceUsed": 1280,
  "totalCalls": 1560
}
```

参数	说明
status	账户状态（ACTIVE/SUSPENDED 等）
serviceUsed	累计成功查得数据的次数（仅统计查得成功 code=001 ）
totalCalls	累计有效查询次数（含查得/查无/部分成功；一次聚合查询计 1 次，不按数据段乘 4；不含被网关拦截的请求）

说明：无任何额度上限拦截，仅做成功查得数统计。

3.3 健康检查

项目	内容
路径	GET /healthz
鉴权	无
响应	HTTP 200, 纯文本 ok

四、返回码说明

4.1 网关返回码 head.errorCode

errorCode	含义	典型原因
0	成功	调用成功（业务结果见 body.code）
505001	appKey 异常	缺少或非法 appKey
505004	账户信息不存在	appKey 未匹配到账户
505007	服务尚未开通	账户停用 / 过期 / 未开通
505002	账号信息异常	签名校验失败
505003	产品编号异常	保留
505062	数据请求异常	参数非法 / 全部数据段获取失败 / 超时未决 / 系统错误（默认错误码）

服务内部处理异常均统一返回 505062，**不计费**，并经异步复查兜底。

4.2 业务结果码 body.code（仅 errorCode = 0 时）

code	含义	四个数据段的状态组合	是否计费
001	查得数据	全部成功应答，且至少一段查得	计费
999	查无结果	全部成功应答，且全部查无	不计费
002	部分数据源成功	至少一段成功应答 + 至少一段获取失败	不计费

002 时 result.range 仍包含已成功段的完整数据与各段状态，调用方可按需使用；因数据不完整，本次查询**不计费**。四段全部失败时不返回 002，而是归一为网关级 505062。

五、计费说明

- 仅当返回 `body.code = 001`（四段全部应答且有查得）时，才计入成功查得数并对客户计费。
- `body.code = 999`（全部查无）**不计费**。
- `body.code = 002`（部分数据段成功）**不计费**——数据不完整不收费，已返回的数据段可正常使用。
- 网关级错误（`head.errorCode` 非 0：鉴权失败、参数非法、全部数据段获取失败、系统异常等）**一律不计费**。
- 计费以最终落库的台账为准，超时未决请求会经异步复查/对账裁定状态，不会重复计费。

六、使用手册（接入与最佳实践）

6.1 接入流程

1. 向商务申请账户，获取 `appKey`、`appSecret`、正式/测试域名。
2. 按第二章实现加签，先在测试环境联调，再切正式环境。
3. 上线后通过成功查得数查询接口（3.2）监控调用量。

6.2 幂等与重试

- 客户端建议为每笔查询设置合理超时（≥ 6 秒；本服务内部并发聚合四个数据段）。
- 收到网络超时/无响应时**可安全重试**：网关基于内部流水号做幂等，不会因重试重复计费。
- 收到 `002`（部分成功）时**可以重试**以尝试获取完整四段数据（`002` 不计费，重试无额外成本；重试后若四段齐全则按 `001` 正常计费）。
- 请勿对已明确返回 `head.errorCode` 的请求做无差别重试（如参数错误 `505062`、鉴权错误 `505001/505002/505004`），应先修正再发起。

6.3 错误处理建议

现象	排查方向
<code>505001 / 505004</code>	检查 <code>appKey</code> 是否正确、是否用错环境账户
<code>505002</code>	检查签名算法（排序/空值剔除/UTF-8/小写 hex）
<code>505007</code>	联系商务确认账户状态与有效期
<code>505062</code>	检查 <code>creditCode</code> 是否为合法 18 位统一社会信用代码；若入参正常仍持续出现，记录 <code>logId</code> 联系我方
频繁返回 <code>002</code>	记录 <code>logId</code> 与 <code>range</code> 中失败段的 <code>error</code> 摘要，联系我方排查对应数据段

任何异常排查请一并提供响应中的 `head.logId`，便于我方全链路定位。

6.4 联调自检清单

- ☐ 域名、appKey、appSecret、环境匹配无误
- ☐ 待签名串严格按 ASCII 升序拼接、剔除空值、UTF-8、MD5 小写
- ☐ creditCode 为合法 18 位统一社会信用代码
- ☐ 能正确解析 head.errorCode 与 body.code 两级状态 (含 002 部分成功分支)
- ☐ 已实现对 result.range 字符串的二次 JSON.parse 解析，并按段处理 ok/empty/error 三种状态
- ☐ 已实现超时重试 (依赖幂等，不重复计费)

附录：术语表

术语	说明
appKey	公开账户标识，随请求明文传输
appSecret	加签密钥，仅本地保存用于计算 sign，切勿泄露或随请求传输
logId	全链路追踪 ID (= head.logId)，排障/对账唯一凭据
creditCode	企业统一社会信用代码 (18 位)，本服务的查询主键
数据段	本服务一次查询聚合的四个数据部分 (invoice1/invoice2/tax1/tax2)
range	四个数据段合并结果的 JSON 字符串，需二次解析 (结构见 3.1.5)